Running LLMs Locally: A Secure Practical Landscape

Mick Cooney

September 2026

Table of Contents

1 Introduction **4**
- 1.1 What This Covers . . . 4
- 1.2 What This Is Not . . . 4
- 1.3 Background Assumed . . . 4
- 1.4 How This Is Structured . . . 4
- 1.5 Quick Read If You Are in a Hurry . . . 5
- 1.6 Privacy Is a Data-Flow Question . . . 5

2 Foundations: What You Are Actually Running **5**
- 2.1 Model Artefact . . . 6
- 2.2 Inference Engine . . . 6
- 2.3 Server Layer . . . 6
- 2.4 Quantisation (the Lever You Will Use Most) . . . 7
- 2.5 Context Window and KV Cache . . . 7
- 2.6 Sampling Controls . . . 8
- 2.7 Decision Models: When You Should Not Generate Text . . . 8
  - 2.7.1 Jev Is Not the Same as a Classifier or Structured Output . . . 8
  - 2.7.2 What Can Run Locally . . . 9
  - 2.7.3 A Practical Local Pattern . . . 10

3 Beyond Text: Choose the Model for the Modality **10**
- 3.1 Practical Model Map . . . 11
- 3.2 What Changed Recently . . . 13
- 3.3 Specialist Models Worth Knowing . . . 13
- 3.4 Image Generation Is Not Vision . . . 13
- 3.5 Running Diffusion Language Models Locally . . . 13
  - 3.5.1 A Practical First Experiment . . . 14
- 3.6 Multimodal Memory Costs . . . 16

4 Hardware Reality in 2026 **16**
- 4.1 CPU-Only Systems . . . 16
- 4.2 NVIDIA GPU Systems . . . 17
- 4.3 AMD GPU Systems . . . 17

4.4 Apple Silicon . . . 17
4.5 Multi-GPU and Multi-Node . . . 18

5 Runtime and Tooling Landscape **18**
5.1 `llama.cpp` and `llama-server` . . . 18
5.2 MLX-LM and MLX-VLM . . . 19
5.3 Ollama . . . 19
5.4 LocalAI . . . 20
5.5 Transformers and Diffusers . . . 20
5.6 `vLLM` . . . 20
5.7 SGLang . . . 21
5.8 LM Studio and Jan . . . 21
5.9 Open WebUI . . . 22
5.10 Comparison Snapshot . . . 22

6 Containers, Runtime Hygiene, and Why Reproducibility Wins **23**
6.1 Why Containers Matter . . . 23
6.2 Docker vs Podman . . . 23
6.3 Image Tags and Drift . . . 24
6.4 Binding Is a Security Decision . . . 24
6.5 GPU Access Notes . . . 24

7 Deployment Blueprints That Work **25**
7.1 Blueprint 1: Personal Chat in Under 20 Minutes . . . 25
7.2 Blueprint 2: Local Coding Assistant in VS Code . . . 25
7.3 Blueprint 3: Persistent Local API Service . . . 26
7.4 Blueprint 4: Multi-Modal Local Platform . . . 26
7.5 Blueprint 5: High-Concurrency API for Teams . . . 27

8 Model Selection in 2026 **28**
8.1 The Selection Order That Works . . . 28
8.2 Workload Shapes . . . 28
8.3 Family-Level Guidance (Practical) . . . 28
8.4 Size and Quantisation Pairing . . . 29
8.5 Where to Source Models . . . 29
8.6 Minimal Local Eval Loop . . . 29

9 API Compatibility and Application Design **30**
9.1 Keep App Code on OpenAI-Compatible Calls . . . 30
9.2 Guardrails for Tool Calling . . . 31
9.3 RAG Layer Decisions . . . 31

10 Operations: Keep It Stable Over Time **32**
10.1 Baseline Checklist . . . 32
10.2 Update Discipline . . . 33
10.3 Observability That Actually Helps . . . 33
10.4 Security and Privacy Reality . . . 33

11 Decision Guide by Constraint **34**
11.1 I Want the Easiest Possible Local Setup . . . 34
11.2 I Want Maximum Low-Level Control . . . 34
11.3 I Need High Concurrency for Team APIs . . . 34
11.4 I Need One Local Platform for Text + Embeddings + Audio + Image . . . 34
11.5 I Need Image Generation or Image Editing . . . 34
11.6 I Need OCR, Screenshots, or Document Understanding . . . 34
11.7 I Need Local Transcription or Speech Generation . . . 34
11.8 I Need Local Video Generation . . . 35
11.9 I Want a Polished Local Chat UI . . . 35
11.10I Want Private Assistant Workflows Through Messaging Apps . . . 35

12 5-Day Practical Starter Path **35**
12.1 Day 1: Run a Local Model . . . 35
12.2 Day 2: Add Browser UI . . . 35
12.3 Day 3: Use the API Directly . . . 35
12.4 Day 4: Containerise and Persist . . . 36
12.5 Day 5: Compare Two Model Families . . . 36

13 Conclusion **36**

14 Appendix **36**
14.1 Quick Command Cheatsheet . . . 36
14.2 Further Reading . . . 37

From first prompt to a maintainable private AI stack

September 2026 · runtime and model references reviewed 30 September 2026

You have used cloud AI tools.

Now you want control, but "local" has to mean more than a model file on your disk. A browser UI can retain chats, a search connector can send prompts away, and an unauthenticated API can be reachable from a network you did not intend to trust.

You want prompts and documents to stay on your machine, predictable cost, and the option to wire models into your own tools without waiting on a vendor roadmap. You also need to understand what is actually running and who can reach it.

Verify compatibility and model cards before standardising a deployment.

1 Introduction

1.1 What This Covers

This is a practical guide to running models locally for chat, coding, API workloads, retrieval, media work, and agent backends. You will learn:

1. The minimum concepts you need to avoid common dead ends.
2. How hardware limits shape model choices.
3. How the major runtime options differ in 2026.
4. Deployment patterns that work in practice.
5. How text, vision, image-generation, speech, audio, and video models differ.
6. How to choose, test, and operate models without guesswork.
7. How to keep personal services loopback-only, and what has to change before you share one.

1.2 What This Is Not

This is not a training guide. It does not teach full fine-tuning pipelines, distributed pretraining, or benchmark archaeology. It does not try to be a complete reference for every inference engine.

It is a decision-and-deployment guide, not a claim that a service becomes safe by calling it private or local. Model names and runtime support move quickly; the dated links and model cards matter more than any static ranking.

1.3 Background Assumed

You can use a terminal, install software, and read logs. If you know what a container is and have used Docker or Podman once, you are set. If you have not, you can still follow along, but expect to pause and look up a couple of commands.

1.4 How This Is Structured

Start with one text model, one user, and a loopback-only endpoint. Branch to a UI, coding assistant, retrieval, multimodal work, or shared serving only when the task needs it. The guide expands that path:

1. Foundations (the concepts that actually matter at runtime)
2. Hardware constraints (what fits and what does not)
3. Inference stack options (what each tool is good at)

4. Deployment blueprints (copyable patterns)
5. Model and operations decisions (how to stay sane over time)

1.5 Quick Read If You Are in a Hurry

If you need one short answer:

1. Start with Ollama if you want speed-to-first-result; keep it on loopback.
2. Use llama.cpp directly if you want control over a GGUF deployment.
3. Use vLLM or SGLang only for measured concurrent serving demand.
4. Use LocalAI when one local API for several modalities is genuinely useful.
5. Use ComfyUI or Diffusers for image and video generation, not a chat model.
6. Keep application code on OpenAI-compatible APIs, behind a small adapter.
7. Before exposing any port, add authentication, authorisation, TLS, a firewall rule, and a named owner for the service.

1.6 Privacy Is a Data-Flow Question

Local inference keeps prompts local only when the whole workflow is local. Trace every arrow, including the ones outside the model runtime:

```
user -> UI or application -> model server -> model files
                          \-> tools, search, telemetry, backups, logs
```

Decide which systems may receive prompts, uploaded files, chat history, retrieved chunks, and logs. Disable or isolate remote search, cloud embeddings, crash reporting, agent connectors, and backups that fall outside that boundary. An offline model with an unrestricted shell tool or a public HTTP port is not a private assistant.

---

2 Foundations: What You Are Actually Running

Most confusion in local LLM work comes from blending three layers together.

Separate them:

1. The model artefact (weights + metadata)
2. The inference engine (token generation runtime)
3. The server or UI layer (APIs, chat interfaces, auth)

When you keep these layers distinct, the ecosystem stops feeling messy.

2.1 Model Artefact

A model is a large set of learned weights plus metadata about tokenizer, architecture, and prompt formatting.

For local inference, you will mostly see:

1. GGUF files for llama.cpp-class runtimes
2. Safetensors/Hugging Face checkpoints for transformer-native and diffusion runtimes
3. MLX-converted weights for Apple Silicon
4. Pipeline components such as a VAE, text encoder, vision encoder, or multimodal projector

GGUF remains the practical default for many local setups because it is portable and quantisation-friendly.

It is not the universal format. Image and video generators are commonly distributed as Safetensors components, while Apple-oriented tools often use MLX conversions. A vision-language model may also need a separate vision tower or `mmproj` file.

2.2 Inference Engine

The engine performs tokenization, forward passes, cache management, and sampling.

At runtime, a single response loop is:

1. Convert text to tokens.
2. Run those tokens through the model.
3. Compute next-token probabilities.
4. Sample one token.
5. Append token and repeat.

This is why memory bandwidth and cache behaviour matter so much. Even with good compute hardware, poor memory fit destroys throughput.

2.3 Server Layer

The server exposes the engine over HTTP and handles request structure, batching, and optional auth.

If you keep to OpenAI-compatible endpoints, you can usually swap runtimes with small config changes.

That API stability is what makes local inference practical for real applications, not just demos.

2.4 Quantisation (the Lever You Will Use Most)

Quantisation reduces weight precision to make models fit smaller hardware.

A simplified mental model:

1. Higher precision: better quality ceiling, more memory, slower on constrained hardware.
2. Lower precision: smaller footprint, faster, but quality loss appears sooner on complex tasks.

Common practical choices:

Quantisation	Typical Use
FP16/BF16	Datacenter or high-memory local GPU runs
8-bit	Quality-sensitive local serving with decent VRAM
4-bit	Common local sweet spot for many personal text-model runs
2-3 bit	Extreme memory constraint, quality trade-off is obvious

In the GGUF world, 4-bit variants like Q4_K_M remain a strong llama.cpp baseline for many assistants and coding workflows. They are not a universal optimum: image/video pipelines and accelerator servers often use different formats, and quantisation quality depends on the model and hardware.

2.5 Context Window and KV Cache

Context window is how much text the model can attend to at once. KV cache is the memory used to store token history state.

Longer context means larger KV cache. That memory cost can dominate runs that otherwise fit fine.

A practical rule:

1. Pick model size and quantisation first.
2. Then increase context until latency and memory stay acceptable.
3. Do not assume max advertised context is affordable on your hardware.

2.6 Sampling Controls

The three settings that matter most:

1. Temperature: randomness.
2. Top-p: probability mass cutoff.
3. Repetition controls: avoid loops and overuse.

For coding and precise factual tasks, run cooler. For brainstorming, increase controlled randomness.

2.7 Decision Models: When You Should Not Generate Text

Many applications do not need an answer written for a person. They need a route, a score, a yes/no judgment, or the next item from a finite set. Using a generative LLM for that job creates an avoidable conversion layer: prompt the model, make it emit a string that resembles a schema, parse the string, handle invalid output, and then decide whether its confidence estimate means anything. A local classifier, natural-language-inference (NLI) model, or ordinary rules may be a better fit.

Jev is the most visible recent example of a different interface. TypeSafe calls it a *System One model*: give it a state and typed questions, and receive structured values plus probability distributions. Its documented primitives are `choice` (select from options), `score` (select an ordered rubric), and `noul` (a yes/no probability). It does not generate a prose answer. The TypeSafe introduction describes the wire-level contract; the launch article describes the company's RLCD (Reinforcement Learning for Calibrated Decisions) training approach and performance claims.

The useful mental model is a probabilistic function call:

```
state + typed question(s) -> typed result(s) + probabilities
```

For example, a support workflow might pass the ticket text as `state` and ask which team should own it, whether the customer is angry, and how urgent it is. The surrounding program can route high-confidence tickets, send ambiguous ones to a larger model or a person, and combine several atomic scores with explicit business logic. That decomposition is important: a decision model is not a general autonomous reasoner, and its confidence does not prove that the question or option set was well designed.

2.7.1 Jev Is Not the Same as a Classifier or Structured Output

These tools overlap, but they are not interchangeable:

Tool	Input	Output	Best default use
Rules or supervised classifier	Features or fixed labels	Label or score	Stable task with labelled data
NLI / zero-shot classifier	Text plus labels	Label scores	Small classification experiments
Structured-output LLM	Context plus a schema	Constrained generated data	Open-ended reasoning with typed output
Jev/System One-style model	State plus options or rubric	Typed result and probabilities	Fast, repeated software decisions
Embedding model	Text, image, audio, or other content	Vector	Retrieval or similarity, not policy

The distinction is architectural rather than just a marketing label. A constrained LLM still performs token generation and can misunderstand the task while producing valid JSON. A decision model can make an incorrect choice while never producing malformed JSON. Type safety removes one failure class; it does not remove semantic error, distribution shift, bad labels, poor calibration, or prompt injection in the state. Treat the returned probability as a signal to validate on your own workload, not as a licence to skip evaluation.

2.7.2 What Can Run Locally

Jev itself is a hosted, proprietary API at the time of this review. The local ecosystem is young and should be treated as a set of experiments, not as a settled compatibility layer. Examples include:

- Laya, an open-weight typed decision model intended for local inference.
- OpenJev, an independent open-weight implementation with `choice`, `score`, and `noul`-style questions.
- Decision-4B, an open-weight adapter based on Qwen3.5-4B with a GGUF route documented for llama.cpp and Ollama.
- Small NLI or encoder classifiers, and a fine-tuned model trained on your own labels, when the task is narrower than a general typed-decision model.

"Open-weight Jev" is shorthand for a similar decision interface or training goal, not proof of equivalence to Jev. Check the model card, licence, maximum number of options, supported languages, input modalities, calibration results, and serving path. A model may expose a

Jev-compatible endpoint while using ordinary autoregressive generation underneath; that can still be useful, but it changes the latency, memory, and output-reliability story. A model card that reports accuracy without calibration or abstention behaviour is not enough for confidence-gated automation.

2.7.3 A Practical Local Pattern

Keep the decision model behind ordinary application code. Do not let it execute tools directly. Validate the question and option set, record the model revision and calibration threshold, and define what happens below that threshold:

```
input -> local decision model
      -> high confidence: ordinary deterministic branch
      -> low confidence: larger LLM, human review, or explicit fallback
```

Start with a labelled hold-out set from the real workflow. Measure accuracy by class, confusion matrix, latency, throughput, and calibration (for example, whether predictions near 0.8 are correct roughly 80% of the time). Include ambiguous, adversarial, out-of-domain, and empty-input cases. Include an explicit `none`, `abstain`, or `review` option when the real task sometimes has no safe answer; forcing every input into a valid label produces clean-looking metrics and unsafe automation. If the task has only a few stable labels and enough historical data, a small supervised classifier will often be cheaper and easier to maintain. If the task needs changing natural-language criteria with little labelled data, a typed-decision model may be a useful middle layer between rules and a full generative LLM.

The decision is therefore workload-shaped. Use a generative model when the system must explain, transform, plan, write code, or handle an open-ended conversation. Use a decision model when the output space is known and the application can express the policy around it. In many useful systems the two are combined: a local decision model handles cheap routing and guardrails, while a larger language model handles only the cases that require generation or deeper reasoning.

---

3 Beyond Text: Choose the Model for the Modality

"Local model" no longer means only a chatbot. The important split is between models that *understand* an input and models that *generate* an output. A vision-language model (VLM) reads an image and returns text or structured data. An image-generation model turns text or reference images into pixels. A speech-recognition model turns audio into text. They are different model classes, with different files, runtimes, and hardware requirements.

Do not assume that a model advertised as multimodal can generate every modality. Most multimodal language models accept text plus images, audio, or video and generate text. Image and video generators usually use diffusion or flow-matching pipelines and are operated through tools such as Diffusers, ComfyUI, or InvokeAI, not a chat endpoint.

3.1 Practical Model Map

Workload	Current local examples	Typical local path	What to expect
Text, coding, reasoning, and agents	Qwen3.5, Gemma 4, Granite 4.2, gpt-oss, LFM2.5	llama.cpp, Ollama, vLLM, SGLang, Transformers, MLX-LM	The broadest ecosystem. Size, quantisation, and context length still dominate fit.
Image understanding, OCR, charts, and documents	Qwen3-VL, Gemma 4, Granite Vision, LFM2.5-VL	Transformers, vLLM, SGLang, Ollama vision models, or supported llama.cpp multimodal builds	These models describe and reason about pixels; they do not create finished images. OCR quality depends heavily on resolution and layout.
Text-to-image and image editing	FLUX.2, Qwen-Image, Stable Diffusion family	Diffusers, ComfyUI, InvokeAI	Treat the pipeline, text encoder, variational autoencoder (VAE), and transformer/diffusion weights as one artefact. VRAM rises quickly with resolution and reference images. Licenses differ by checkpoint.

Workload	Current local examples	Typical local path	What to expect
Speech recognition and alignment	Whisper, faster-whisper, Qwen3-ASR, Granite Speech	faster-whisper/CTranslate2, Transformers, vLLM, or experimental llama.cpp audio support	Usually much smaller and easier to run than a general LLM. Use a dedicated automatic speech recognition (ASR) model when transcription is the job.
Speech synthesis and audio understanding	Qwen3-Omni, XTTS v2, CosyVoice	Transformers, project-specific runtimes, or LocalAI	Text-to-speech (TTS), voice cloning, and audio captioning have different quality and consent risks. Check speaker-data terms before cloning a voice.
Video understanding and generation	Qwen3-VL, Qwen3-Omni, Wan2.2	vLLM, SGLang, or Transformers for understanding; Diffusers or model code for generation	Video generation is substantially more demanding than image generation. Wan2.2's 5B text-image-to-video path documents 24 GB VRAM at 720p; its larger paths need much more.
Text and multimodal retrieval	EmbeddingGemma, Qwen3-VL-Embedding and Reranker, BGE-family models	Sentence Transformers, Transformers, Ollama, or a vector database	Embeddings produce vectors, not answers. Evaluate retrieval separately from the language model that writes the final response.

3.2 What Changed Recently

The September 2026 snapshot points towards smaller specialists, sparse larger models, configurable reasoning, native tool calling, multimodal input, and long-context variants. Qwen3.5, Granite 4.2, and Gemma 4 are useful examples, not a permanent shortlist. Parameter count alone is no longer a fit estimate: active parameters, modality encoders, context length, quantisation, and draft-model support all affect actual memory and throughput.

3.3 Specialist Models Worth Knowing

For task selection, use a compact text/coding model first; use a VLM or OCR specialist for document structure; use ASR/TTS models for speech; and use a dedicated diffusion or flow pipeline for image/video generation. Current examples include FLUX.2 or Qwen-Image for pixels, PaddleOCR-VL or GLM-OCR for layout-aware extraction, and Whisper or Qwen3-ASR for transcription. These are orientation points, not a catalogue: verify the exact artefact, runtime, hardware fit, and licence. Voice cloning requires speaker consent; video generation remains a workstation-scale workload.

3.4 Image Generation Is Not Vision

This distinction prevents a common failed setup. A VLM such as Qwen3-VL can inspect a screenshot, read a receipt, or answer questions about a photograph. An image model such as FLUX.2 or Qwen-Image can synthesise or edit pixels. They may share a text encoder or appear in the same UI, but they are not interchangeable. For a private document pipeline, the usual sequence is:

1. Use a VLM or OCR model to extract structure from the document.
2. Use a text model to classify, summarise, or call tools.
3. Use an image model only if the workflow needs a generated or edited image.

The same separation applies to audio and video. Transcribe with ASR, reason over the transcript with a language model, and synthesise speech or video only at the output stage when required.

3.5 Running Diffusion Language Models Locally

Diffusion language models use the same broad idea as image diffusion — start from a noisy or masked state and refine it — but they need a different inference loop from an ordinary causal LLM. A model such as LLaDA predicts masked token positions over several refinement steps. That means a normal "load a Transformers model and call `generate()`" recipe is not enough:

you need the model's sampler, masking rules, and stopping logic as well as its weights and tokenizer.

The software stack has four pieces:

1. **Model checkpoint.** The official LLaDA releases use Hugging Face transformer checkpoints, usually Safetensors plus custom model code. Quantised local deployments may instead use GGUF. Do not assume that a GGUF made for a causal Llama model will work; the diffusion architecture and sampler must be supported by the runtime.
2. **Diffusion-aware inference engine.** The official LLaDA PyTorch implementation is the most direct reference path. Current llama.cpp diffusion support includes LLaDA and Dream examples, while diffuse-cpp provides a portable C++ route for supported GGUF checkpoints.
3. **Accelerator backend.** PyTorch normally uses CUDA on NVIDIA, ROCm on supported AMD cards, or Metal/MPS on Apple hardware. llama.cpp has its own CPU, CUDA, Metal, Vulkan, and other backend paths. A model working in one backend does not prove that its diffusion sampler works in another.
4. **Optional server or UI.** Start with a CLI while validating the model. Add an OpenAI-compatible server only after you know how the runtime represents diffusion steps, maximum output length, batching, and cancellation. A chat UI can hide those details and make a slow or incorrect sampler look like a model-quality problem.

3.5.1 A Practical First Experiment

For a desktop or workstation, start with an 8B instruction-tuned checkpoint and a 4-bit GGUF. One published LLaDA Q4_K_M conversion is about 5.1 GB; its model card also lists an approximately 15 GB FP16 variant. The file size is only the first bound: leave room for the runtime, activations, tokenizer, context, and any cache or scratch buffers. A 4-bit checkpoint is a sensible starting point for a machine with roughly 8–16 GB of usable GPU memory, or more system RAM if running mostly on the CPU.

After installing a current llama.cpp build with its diffusion executable and downloading a compatible GGUF, the shape of a first run is:

```
# Example model download; inspect the model card before accepting its licence.
huggingface-cli download diffuse-cpp/LLaDA-8B-Instruct-GGUF \
  llada-8b-q4km.gguf --local-dir ./models/llada-8b

# Run the diffusion-specific CLI. The exact binary name can vary by build.
llama-diffusion-cli \
  -m ./models/llada-8b/llada-8b-q4km.gguf \
  -p "Explain why a Kalman filter works" \
  -ub 512 \
```

```
  --diffusion-block-length 32 \
  --diffusion-steps 128
```

The step count is not equivalent to "maximum new tokens." It controls how many refinement passes the sampler makes; block length, confidence or entropy selection, temperature, and output length are separate controls. More steps can improve convergence but increase latency. Measure time to first usable answer, total completion time, tokens or positions per second, and peak memory rather than comparing only nominal parallelism.

The official PyTorch route is useful when you want the reference implementation or intend to modify the sampler:

```
python -m venv .venv
source .venv/bin/activate
pip install torch transformers==4.38.2
git clone https://github.com/lrbob/LLaDA
cd LLaDA
python chat.py
```

Use the repository's current instructions for checkpoint selection, CUDA or MPS installation, and model loading. BF16/FP16 inference for an 8B model generally wants a high-memory GPU; CPU execution is possible but usually much slower and may require substantial system RAM. Quantisation reduces weight memory, not all working memory, and an MoE model's "active parameters" describe compute per step, not necessarily the total checkpoint that must be stored.

For larger or newer families, use a runtime that explicitly names the model. dInfer supports LLaDA, LLaDA-MoE, and LLaDA2 through specialised inference paths, with vLLM and SGLang backends depending on the model family. This is a better fit for a CUDA workstation or multi-GPU server than for a first laptop experiment. Treat support as model-specific: LLaDA, LLaDA-MoE, LLaDA2, and Dream do not automatically share the same checkpoint format, sampler flags, or serving API.

GPU memory has three separate costs:

1. **Weights:** reduced by 4-bit or 8-bit quantisation.
2. **Working memory:** activations, temporary logits, tokenizer buffers, and the repeated denoising passes.
3. **Context and batching:** attention/cache state and simultaneous requests; these grow with sequence length and batch size.

Diffusion can do more positions in parallel, but it may run many full model evaluations. That can make it compute-bound where a causal LLM is limited by serial token generation or memory bandwidth. GPU offload helps only when the backend supports the architecture well; spilling layers to CPU can make a nominally fitting model unpleasantly slow. On Apple

Silicon, unified memory can let the model fit without a discrete GPU, but the CPU and GPU share that memory. On AMD, Vulkan or ROCm support depends on the exact runtime and card. Validate the smallest supported checkpoint end to end before planning around a larger one.

3.6 Multimodal Memory Costs

Images, audio, and video become tokens or latent features before the language model sees them. A single high-resolution image or a sampled video can therefore consume more context and key-value (KV) cache than its file size suggests. Start with low resolution, short clips, and a small number of images; increase them only after measuring memory and latency. A model that fits text-only may fail once its vision tower, projector, audio encoder, or video loader is enabled.

---

4 Hardware Reality in 2026

Hardware constraints are still the biggest determinant of user experience.

4.1 CPU-Only Systems

CPU-only inference is valid, but expectations matter.

CPU-only is good for:

1. Small models
2. Batch embedding jobs
3. Offline or low-interactivity pipelines

CPU-only is not ideal for:

1. Large interactive chat at long context
2. Multi-user concurrent serving
3. Tool-heavy agent loops with strict latency needs

Modern llama.cpp builds are excellent on CPU, but a slow token stream is still a slow token stream.

4.2 NVIDIA GPU Systems

NVIDIA remains the easiest path for predictable high-performance local inference because CUDA tooling is mature across runtimes.

Approximate fit guidance for mainstream quantised models:

VRAM	Comfortable Tier
6-8 GB	Small 3B-8B models at 4-bit
12-16 GB	8B-14B class at 4-bit, some 8-bit options
24 GB	30B class at 4-bit in many setups
48 GB+	70B class at 4-bit for serious local serving

Exact fit depends on runtime, context, and architecture. Always leave memory headroom for KV cache and framework overhead.

4.3 AMD GPU Systems

ROCm is now materially better than it was two years ago. Many local users run AMD successfully in 2026, and Vulkan paths are useful for some consumer cards and applications. Intel XPU support is also present in parts of the serving ecosystem.

Still, setup friction can be higher than CUDA depending on distro, card generation, and runtime support level. Backend support is not uniform: a model may work in llama.cpp on ROCm but not in a particular vLLM or SGLang build, or may support image inputs only on one backend.

Use AMD when:

1. Your target runtime documents strong ROCm support.
2. You are willing to spend time validating kernel and driver combinations.

Avoid wishful assumptions. Test your exact card + runtime pair early.

4.4 Apple Silicon

Apple Silicon remains one of the best personal platforms for local inference due to unified memory and strong Metal acceleration.

The key trade-off is that very large models are memory-bandwidth sensitive. You can run them, but throughput can flatten faster than people expect.

For many users, a well-chosen 8B-30B class model on Apple Silicon provides the best quality-to-latency balance. MLX and `mlx-lm`/`mlx-vlm` now make Apple-native text and selected vision workloads particularly attractive; model-specific conversion still matters.

4.5 Multi-GPU and Multi-Node

Splitting models across multiple GPUs or nodes is possible and increasingly common in hobby homelabs.

Do it only when you need it. Complexity rises quickly:

1. More failure modes
2. Harder reproducibility
3. More debugging surface in networking and scheduling

If one larger GPU solves your problem, it is often cheaper in time than distributed experimentation.

---

5 Runtime and Tooling Landscape

There are more choices now, but the core categories are stable.

At the 10 September 2026 cutoff, the release pages still showed a fast-moving stack across Ollama, vLLM, SGLang, MLX, and Transformers. This document deliberately does not pin a "current" version: runtime and model compatibility is more important than the newest tag, and a moving version number ages immediately. Read the model recipe and accelerator requirements before upgrading.

5.1 `llama.cpp` and `llama-server`

llama.cpp is still the foundational local inference engine for GGUF workflows.

Use it when you want:

1. Fine-grained control over runtime flags
2. Strong CPU performance
3. Portable execution across many hardware backends

`llama-server` gives you an OpenAI-compatible API endpoint plus basic web chat. Its current multimodal path uses `libmtmd` and supports image and experimental audio input for selected architectures. These models commonly need two GGUF files: the language model and a

matching multimodal projector (`mmproj`). Check the multimodal support list; a normal GGUF file is not automatically vision-capable.

```
# Serve a model pulled from Hugging Face through llama.cpp, locally only.
llama-server -hf ggml-org/gemma-3-1b-it-GGUF --host 127.0.0.1 --port 8080

# Or serve a local GGUF file, locally only.
llama-server -m ./model.gguf --host 127.0.0.1 --port 8080
```

`127.0.0.1` is deliberate. `--host 0.0.0.0` changes the service from a personal endpoint into a network-reachable one. Do not use it until the gateway, authentication, TLS, firewall, and access policy described later are in place. If you enjoy tuning and understanding the engine, llama.cpp remains a top choice.

5.2 MLX-LM and MLX-VLM

MLX is Apple's array framework for Apple Silicon. `mlx-lm` is a strong path for text models on Macs, and `mlx-vlm` adds model-specific vision-language support. These tools exploit unified memory and avoid the CUDA assumptions in many server stacks. They are excellent for a Mac laptop or workstation, but model conversion and support are more model-specific than with Transformers.

For image generation, use an MLX-specific implementation only when the model's repository documents it. Otherwise use a supported Diffusers or ComfyUI pipeline and expect higher memory pressure.

5.3 Ollama

Ollama is still the fastest path from zero to a working local model, but it is no longer text-only in practice. Current releases support selected vision models, image inputs, tools, reasoning controls, structured output, and an OpenAI-compatible Responses API. On Apple Silicon, selected models can use an MLX-backed path.

Use it when you want:

1. Straightforward model pull/run lifecycle
2. Local API with minimal setup
3. Good integration with coding tools and chat frontends

```
# Install on Linux
curl -fsSL https://ollama.com/install.sh | sh

# Run a model immediately
ollama run gemma4

# Start API service
ollama serve
```

Ollama keeps improving ergonomics and model catalogue experience. It is not the most configurable engine, and its OpenAI-compatible surface is only a subset: base64 image data works, while remote image URLs and some tool controls do not. It remains the most productive default for a single user who wants to try text or vision models quickly.

5.4 LocalAI

LocalAI is a broad local AI platform exposing OpenAI-style interfaces across text, embedding, audio, and image workflows through multiple backends.

Use it when you want:

1. One local API surface for multiple modalities
2. Flexible backend selection
3. Container-first operations

```
# CPU example: publish only to the local machine.
podman run -d --name localai -p 127.0.0.1:8080:8080 localai/localai:<tested-tag>

# NVIDIA example (verify current image tag in LocalAI docs)
podman run -d --name localai -p 127.0.0.1:8080:8080 --gpus all \
  localai/localai:<tested-nvidia-tag>
```

The `127.0.0.1:` prefix matters: `-p 8080:8080` commonly publishes the port on every host interface. It is powerful and flexible, but there is more operational surface area than Ollama. LocalAI is a useful compatibility layer, not proof that every underlying model has equal support or quality. Validate each modality separately.

5.5 Transformers and Diffusers

Transformers remains the model-definition and native-inference layer for many VLM, OCR, ASR, and text models. Its newer `transformers serve` command can expose an OpenAI-compatible endpoint for supported models, which is convenient for testing before moving to vLLM or SGLang.

Diffusers is the corresponding practical layer for image and video generation. It handles pipelines, schedulers, VAEs, adapters, and quantisation. Use it for Python integration and reproducibility; use ComfyUI when you want to inspect and iterate on a graph interactively.

5.6 `vLLM`

vLLM remains one of the leading choices for high-throughput serving with strong batching behaviour and efficient cache management. Its supported-model surface now includes text, image, video, and audio inputs for selected architectures, with the exact combination depending on the model and backend.

Use it when you want:

1. Better concurrent throughput than simple single-request loops
2. Production-oriented serving behaviour
3. Strong support for modern large-model serving patterns

```
# Python install route
uv pip install vllm
vllm serve meta-llama/Llama-3.3-8B-Instruct
```

For local single-user chat, vLLM can be overkill. For multi-user APIs, it is often the right level of machinery. Its main quantisation path is Hugging Face-native (for example FP8, GPTQ, AWQ, MXFP4, NVFP4, and quantised KV cache); GGUF support is experimental and requires the separate vLLM GGUF plugin.

5.7 SGLang

SGLang is now a serious option for optimised LLM, multimodal, and diffusion serving, especially where structured generation and advanced decode/scheduling behaviour matter. Its support is accelerator- and backend-specific, so verify the model's recipe before treating a claimed feature as portable.

Use it when you want:

1. Competitive serving performance in modern accelerator stacks
2. Advanced control for structured and multimodal output workloads
3. Another mature path beyond vLLM for high-demand deployments

In practice, serious teams often benchmark both vLLM and SGLang for their exact request patterns before committing. For image or video generation, SGLang-Diffusion is a separate path from ordinary chat serving.

5.8 LM Studio and Jan

LM Studio and Jan remain useful GUI-first desktop choices.

Use them when:

1. You want model experimentation without shell-heavy setup
2. You prefer desktop UX over service administration
3. You still want local APIs for integrations

These tools are excellent for exploration and personal workflows, less so for hardened shared server operations.

5.9 Open WebUI

Open WebUI is still the dominant self-hosted chat frontend in this ecosystem. It is becoming an agent frontend as well, with tool approval and richer state handling. Treat it as a versioned application, not an unpinned image.

Use it when you need:

1. Browser chat UI
2. Multi-user accounts
3. RAG-style document chat features on top of your backend

```
podman run -d -p 127.0.0.1:3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  -v open-webui:/app/backend/data \
  ghcr.io/open-webui/open-webui:<tested-tag>
```

It does not replace inference backends. It sits in front of them. Create the initial administrator deliberately, disable self-registration unless you need it, and review web search, tools, connectors, upload retention, and outbound network settings before giving anyone access. Check the release page before copying the version tag, and keep it patched: an earlier 2026 release line had a DNS-rebinding security advisory.

5.10 Comparison Snapshot

Tool	Easiest Start	High Control	High Throughput	Multi-Modal Scope	Typical Persona
Ollama	Excellent	Medium	Medium	Text + selected vision/tools	Solo dev, fast setup
llama.cpp	Medium	Excellent	Medium	Text + selected image/audio input	Tuner, systems-minded user
LocalAI	Medium	High	Medium-High	Broad	Platform builder
vLLM	Low	High	Excellent	Text + selected media	API team

Tool	Easiest Start	High Control	High Throughput	Multi-Modal Scope	Typical Persona
SGLang	Low	High	Excellent	Text + media + diffusion	Performance-focused team
MLX-LM + MLX-VLM	Medium	High on Apple	Medium	Text + selected vision models	Apple Silicon user
Transformers + Diffusers	Medium	Excellent	Medium-High	VLM/ASR + image/video generation	Python integrator
LM Studio / Jan	Excellent	Low-Medium	Low	Personal use	GUI-first user
Open WebUI	High (as UI)	N/A	N/A	UI layer	Team chat frontend

---

6 Containers, Runtime Hygiene, and Why Reproducibility Wins

Containerisation is a strong default for Linux server-style local inference. It improves reproducibility; it does not make an exposed HTTP service private.

6.1 Why Containers Matter

You isolate dependencies:

1. CUDA/ROCm stacks
2. Python and native library versions
3. Runtime binaries and model-serving flags

This makes rollback possible and debugging less chaotic.

6.2 Docker vs Podman

Both are viable. Podman remains attractive for rootless operation and daemonless workflow. Docker still has broader tutorial gravity.

Pick one and standardise across your own docs and scripts. The consistency is more important than the brand.

6.3 Image Tags and Drift

Never treat `latest` as a long-term contract.

For personal experimentation, `latest` is fine.

For repeatable environments, pin tags or digests and record:

1. Runtime image
2. Model identifier and revision
3. Key serving flags

That single discipline prevents many "it worked last week" incidents.

6.4 Binding Is a Security Decision

A container port published as `-p 8080:8080`, a runtime bound to `0.0.0.0`, or a router port-forward all create a network service. Do not copy those forms from a tutorial as a convenience setting. For a personal stack, bind the host side explicitly to loopback:

```
podman run -d --name local-api \
  -p 127.0.0.1:8080:8080 \
  -v model-data:/models \
  <tested-image>
```

For a shared service, keep the inference runtime on a private network or loopback and expose only an authenticated reverse proxy or gateway. Terminate TLS there, authenticate every caller, authorise models/uploads/tools by user or service, and firewall the gateway to intended networks. A placeholder API key that a client library insists on is not authentication. Keep real credentials out of command lines, images, prompts, repositories, and browser JavaScript; use a protected secret mechanism and rotate them.

6.5 GPU Access Notes

GPU passthrough is still the most common deployment failure point.

Checklist:

1. Validate host driver stack first.
2. Validate container toolkit runtime access second.
3. Only then debug model/server flags.

If you invert this order, you lose hours in application-level logs for a host-level problem.

7 Deployment Blueprints That Work

These are opinionated starting points that fit common goals.

7.1 Blueprint 1: Personal Chat in Under 20 Minutes

Use Ollama + optional Open WebUI.

```
curl -fsSL https://ollama.com/install.sh | sh
# Choose a currently published compact instruct model after reading its card.
ollama run <small-instruct-model>

# Verify that the service remains local before connecting a UI or IDE.
ss -ltnp | rg '11434|ollama'
curl http://127.0.0.1:11434/api/tags
```

Then add Open WebUI if you want browser chat and conversation history, still bound to `127.0.0.1`. Treat chat history, uploaded documents, and enabled web or tool connectors as data stores and capabilities that need an explicit retention and approval policy.

Who this is for:

1. First local setup
2. Privacy-first personal usage
3. No strict performance constraints

7.2 Blueprint 2: Local Coding Assistant in VS Code

Use Ollama as backend and connect via Continue or other OpenAI-compatible extension.

Example `~/.continue/config.yaml`:

```
models:
  - name: Local coding chat
    provider: ollama
    model: qwen3-coder:8b
    roles:
      - chat

  - name: Local autocomplete
    provider: ollama
    model: qwen3-coder:1.5b
    roles:
      - autocomplete
```

If tag names differ in your environment, pick the nearest available coder variants from your local registry.

Who this is for:

1. Daily coding assistance
2. Fast edit-feedback loops
3. Strong local privacy boundary

7.3 Blueprint 3: Persistent Local API Service

Use `llama-server` or Ollama in a container with persistent volumes and service management.

For Linux + Podman + user systemd, prefer Quadlet units.

`~/.config/containers/systemd/llm-api.container`:

```
[Unit]
Description=Local LLM API

[Container]
Image=ghcr.io/ggml-org/llama.cpp:server-cuda
ContainerName=llm-api
PublishPort=127.0.0.1:8080:8080
AddDevice=nvidia.com/gpu=all
Exec=-hf ggml-org/Meta-Llama-3.3-8B-Instruct-GGUF --host 127.0.0.1 --port 8080

[Install]
WantedBy=default.target
```

```
systemctl --user daemon-reload
systemctl --user enable --now llm-api.service
```

Who this is for:

1. Stable local endpoint for tools/scripts
2. Home lab service-style operation
3. Repeatable restarts and updates

This is intentionally a single-machine service. A user systemd unit makes restart behaviour repeatable; it does not add authentication. Record the image digest, model revision, quantisation, context limit, and baseline latency alongside the unit. If another machine needs access, put an authenticated TLS gateway in front of it rather than changing both bindings to `0.0.0.0`.

7.4 Blueprint 4: Multi-Modal Local Platform

Use LocalAI when you want one endpoint across text, embeddings, speech, and image workflows. For serious image/video generation, pair a dedicated Diffusers or ComfyUI service with your language-model API rather than forcing every modality through one backend.

```
podman run -d --name localai -p 127.0.0.1:8080:8080 \
  --device nvidia.com/gpu=all \
  -v localai-models:/build/models \
  localai/localai:<tested-nvidia-tag>
```

Add models incrementally and validate each modality independently before composing full pipelines.

Who this is for:

1. Prototype platform teams
2. Self-hosted private AI stacks
3. Mixed modality requirements

Keep the model boundaries visible: VLM/OCR for perception, ASR/TTS for speech, diffusion/flow pipelines for image and video generation, and an LLM for language reasoning. A single frontend can hide these services, but it cannot remove their different memory, licensing, and failure modes.

7.5 Blueprint 5: High-Concurrency API for Teams

Use vLLM or SGLang behind a gateway and benchmark realistic prompt lengths and concurrency. Mixed long prompts and simultaneous requests can favour continuous batching and deliberate KV-cache scheduling.

Who this is for:

1. Internal multi-user applications
2. Latency-sensitive APIs under load
3. Throughput-first architecture decisions

Operational baseline:

1. An authenticated, TLS-terminating gateway; the model runtime remains on a private network.
2. Firewall rules permitting only the proxy or intended subnet; no consumer router port forwarding.
3. Per-user/service authorisation, rate limits, context/output caps, and quotas for expensive models.
4. Structured request logging with protected retention, queue depth, first-token latency, throughput, and error metrics.
5. Per-model configuration profiles and a rollback plan for model/runtime updates.

Do not expose a tool-enabled model server directly to a LAN. Tool definitions, retrieved documents, and agent traces can be as sensitive as the prompt. Give the runtime least privilege, use allowlists and explicit approval for side-effecting tools, and keep credentials unavailable to the model process unless a narrowly scoped tool actually needs them.

8 Model Selection in 2026

Do not choose by hype. Choose by workload.

8.1 The Selection Order That Works

Use this sequence:

1. Define the modality and workload shape; if the output is a bounded decision, test a classifier or typed-decision model before reaching for generation.
2. Pick a family with a licence that fits your use.
3. Pick size for hardware, including encoders and KV cache.
4. Pick a runtime and its supported format.
5. Pick quantisation or an official compressed checkpoint.
6. Run short evals.
7. Freeze the baseline.

8.2 Workload Shapes

Four broad categories capture most local usage:

1. General assistant chat and summarization
2. Coding and tool-use agent workflows
3. Reasoning-heavy long-context tasks
4. Perception or generation of images, audio, and video

Different families and sizes win in different categories. There is no universal best model.

8.3 Family-Level Guidance (Practical)

Use a dated shortlist only to form candidates. At this review date, Qwen, Gemma, Granite, Mistral, gpt-oss, LFM, and established distilled reasoning families cover useful combinations of compact text/coding, tool use, vision, and permissive licensing. Older Llama and Mistral releases remain valuable for compatibility and mature quantised tooling. Do not select by family name: check the exact model card, licence, format, context support, and a realistic evaluation on your hardware. A sparse model's active-parameter count is not its whole memory footprint, and a headline long context may not fit its KV cache on your machine.

8.4 Size and Quantisation Pairing

Use realistic tiers:

Hardware Tier	Good Starting Pair
4-8 GB VRAM / low-memory systems	3B-8B at 4-bit
12-16 GB VRAM	8B-14B at 4-bit, selective 8-bit
24 GB VRAM	14B-30B at 4-bit
48 GB+ VRAM	30B-70B class options

Start smaller than your maximum fit. Fast feedback usually beats marginal quality gains from oversized models.

Those tiers describe text-model weights. Specialist models can be much smaller: 0.9B OCR models and 0.6B ASR models fit where a general VLM does not. Conversely, a 20B image generator, a vision encoder, or a video pipeline can exceed the memory of a similarly sized text model. For image/video generation, use the model's own hardware guidance; for example, Wan2.2 documents 24 GB VRAM for its 5B 720p text-image-to-video path.

8.5 Where to Source Models

Primary sources remain:

1. Hugging Face Hub for broad model and quantisation availability
2. Ollama library for curated pull-and-run simplicity

Before downloading:

1. Check licence fit for your use case.
2. Choose instruct/chat variants unless you need base checkpoints.
3. Confirm file format compatibility with your runtime.
4. Record exact model identifiers used in your stack.

For visual and audio models, also record the encoder/projector, pipeline version, resolution or clip length, and any safety or watermarking defaults. For voice cloning and image-generation checkpoints, record the usage restrictions separately from the code licence.

8.6 Minimal Local Eval Loop

Do not trust one impression prompt. Build a tiny repeatable eval set.

Include 15-30 prompts that represent your real work:

1. One-turn instruction following
2. Multi-turn memory behaviour
3. Coding edits or debugging responses
4. Tool-call formatting correctness
5. Domain-specific reasoning checks

Score each candidate for:

1. Accuracy
2. Latency
3. Stability
4. Cost-to-run on your hardware

That small harness pays off immediately.

---

9 API Compatibility and Application Design

You protect future flexibility by designing around stable interfaces.

9.1 Keep App Code on OpenAI-Compatible Calls

Most local runtimes expose chat-completions-style APIs.

Do not force every local model through a chat-completions abstraction. A typed-decision endpoint has a different contract: state, typed questions, probabilities, and calibration metadata rather than messages and generated tokens. Put it behind a small application adapter alongside the text-model client. This lets a workflow route simple cases locally and escalate only the cases that need a generative model, without pretending the two interfaces have the same semantics.

If your app isolates model client config behind environment variables, you can switch backend without rewriting business logic.

The common shape is useful, but compatibility is not identity. Vision requests may require base64 image data, a model-specific content schema, or a separate projector. Audio transcription and text-to-speech normally use different endpoints. Responses APIs, stateful conversations, tool choice, structured output, and remote image URLs also vary by runtime. Build a small adapter around these differences instead of scattering backend-specific assumptions through your application.

Python example:

```python
from openai import OpenAI

client = OpenAI(
    base_url="http://127.0.0.1:11434/v1",
    # A non-empty placeholder can satisfy a client library on loopback only.
    # It is not a credential and must never be used for a shared endpoint.
    api_key="local-placeholder-not-authentication",
)

response = client.chat.completions.create(
    model="qwen3-coder:8b",
    messages=[
        {"role": "system", "content": "You are a concise coding assistant."},
        {"role": "user", "content": "Explain what a KV cache does."},
    ],
    temperature=0.2,
)

print(response.choices[0].message.content)
```

9.2 Guardrails for Tool Calling

If you are building agents, the model chooses a proposed action; it must not hold unlimited authority to execute it. Put the policy boundary in ordinary software outside the model:

1. Validate JSON/tool schemas strictly.
2. Bound retries and tool loops.
3. Log tool calls and outputs with IDs.
4. Separate model mistakes from tool/runtime mistakes in logs.
5. Allowlist tools and arguments, require explicit approval for destructive or external actions, and run tools with least-privilege credentials.

For example, a coding assistant may read the current repository and propose a patch, but a wrapper should reject paths outside the workspace and require a human approval before a deployment command. A retrieval assistant may query an approved collection, but it should not be handed raw database credentials or a general web-fetch tool by default. This saves enormous debugging time once workflows grow beyond toy examples, and it prevents a persuasive model response from becoming an unreviewed system action.

9.3 RAG Layer Decisions

For private document workflows, RAG is still the standard pattern:

1. Chunk documents.
2. Generate embeddings.
3. Store vectors.

4. Retrieve relevant chunks at query time.
5. Inject context into generation prompts.

Common local embedding options remain practical through Ollama and open embedding models hosted on Hugging Face. The newer Qwen3-VL embedding and reranker models extend this pattern to shared text-image-video retrieval, while Google's EmbeddingGemma is a compact text-retrieval option.

Keep retrieval evaluation separate from generation evaluation. Mixing both in one score hides bottlenecks.

Make the boundary visible in the implementation. Store original files, extracted text, embeddings, vector indexes, metadata, and query logs with the same access controls as the source documents. A vector store is not harmless just because it does not contain prose: embeddings, filenames, chunk metadata, and retrieved passages can disclose sensitive information. Test a known-answer set in two stages: first whether the intended chunks were retrieved, then whether the model produced a supported answer and cited or surfaced the right context.

For scanned PDFs, do not assume text extraction is enough. A practical multimodal pipeline can use a specialist OCR/document model such as PaddleOCR-VL, GLM-OCR, or olmOCR, then embed the resulting text and layout. Test tables, formulas, handwriting, reading order, and low-quality scans as separate cases.

---

10 Operations: Keep It Stable Over Time

Most local deployments fail from drift, not from initial setup.

10.1 Baseline Checklist

For each deployed model/runtime combo, record:

1. Runtime image tag or digest
2. Model ID + revision/tag
3. Key inference parameters
4. Context length
5. Hardware target
6. Expected latency envelope

Store this next to your deployment config.

10.2 Update Discipline

Use a simple release procedure:

1. Pull new runtime/model in staging profile.
2. Run your mini eval suite.
3. Compare quality and latency against baseline.
4. Promote only if metrics and behaviour are acceptable.

Local stacks feel "small," but this discipline is still worth it.

10.3 Observability That Actually Helps

You do not need enterprise observability to get value.

Start with:

1. Request ID + model ID in logs
2. Prompt token count, output token count, duration
3. Error rate by endpoint and model

Those three signals usually pinpoint where quality or latency regressed.

For a shared service, add queue time, time to first token, completion throughput, memory use, authenticated caller or service identity, and model revision. Do not default to logging raw prompts, uploaded files, retrieved chunks, or tool output. If debugging requires them, restrict access and define a short retention period before collecting them.

10.4 Security and Privacy Reality

Local inference improves privacy posture. It does not automatically make your stack secure. Secure API endpoints, frontend authentication, tool execution permissions, secrets, backups of chats/vectors, and outbound integrations.

If you expose a service over a network, treat it like any other internal API: use TLS, authentication, authorisation, firewall rules, rate limits, bounded request sizes, and a defined owner. Verify listener addresses after every runtime or container change. `0.0.0.0`, `*:port`, and an unqualified container port publish are warning signs that need an explicit deployment decision, not defaults to leave unexplained.

---

11 Decision Guide by Constraint

Use this when you need a fast recommendation.

11.1 I Want the Easiest Possible Local Setup

Use Ollama first.

11.2 I Want Maximum Low-Level Control

Use llama.cpp directly.

11.3 I Need High Concurrency for Team APIs

Benchmark vLLM and SGLang for your request pattern.

11.4 I Need One Local Platform for Text + Embeddings + Audio + Image

Use LocalAI for a single API surface, but validate each modality independently. For best generation workflows, use LocalAI or an LLM server alongside ComfyUI/Diffusers.

11.5 I Need Image Generation or Image Editing

Start with FLUX.2 Klein 4B or Qwen-Image/Edit if their licenses fit your use. Use ComfyUI for interactive graphs or Diffusers for code. Do not select a VLM just because it accepts images.

11.6 I Need OCR, Screenshots, or Document Understanding

Start with a small VLM or OCR specialist such as Qwen3-VL, Gemma 4, PaddleOCR-VL, GLM-OCR, or olmOCR. Test layout, tables, formulas, and reading order—not only plain paragraphs.

11.7 I Need Local Transcription or Speech Generation

Use Whisper/faster-whisper for broad transcription compatibility, Qwen3-ASR for a newer multilingual ASR option, and Kokoro or Qwen3-TTS for speech synthesis. Voice cloning needs explicit speaker consent and a licence review.

11.8 I Need Local Video Generation

Start with Wan2.2 TI2V-5B if you have around 24 GB VRAM and can accept slow generation. Larger Wan2.2 or synchronised audio-video models move into workstation and multi-GPU territory. Check territorial restrictions on Tencent models before adopting them.

11.9 I Want a Polished Local Chat UI

Use Open WebUI with your backend, or desktop-first tools like LM Studio or Jan.

11.10 I Want Private Assistant Workflows Through Messaging Apps

Use a gateway layer such as OpenClaw in front of your local inference stack. For a deep setup and hardening walkthrough, see the OpenClaw Primer.

---

12 5-Day Practical Starter Path

If you are new and want momentum without chaos, follow this path.

12.1 Day 1: Run a Local Model

Install Ollama and chat with a compact current instruct model. The command is deliberately a placeholder because model tags move faster than this document.

```
curl -fsSL https://ollama.com/install.sh | sh
ollama run <small-instruct-model>
ss -ltnp | rg '11434|ollama'
```

Choose the model from the current runtime library after reading its card and licence. Confirm that the listener is `127.0.0.1` or `[::1]`, not `0.0.0.0` or `*:11434`, before adding anything else.

12.2 Day 2: Add Browser UI

Run Open WebUI and connect it to Ollama on loopback. Decide whether you want chat retention, document uploads, web search, or tools before enabling them.

12.3 Day 3: Use the API Directly

Send chat-completions requests and confirm you can call your local model from code.

12.4 Day 4: Containerise and Persist

Move your runtime into containers with persistent volumes and service startup.

12.5 Day 5: Compare Two Model Families

Run your mini eval prompts across two families and pick one baseline model for your real workflow.

At this point, you are no longer experimenting blindly. You have a repeatable local AI workflow.

---

13 Conclusion

Running open models locally in 2026 is no longer a niche hobby. It is a practical software capability, and the useful unit is no longer always a language model: it may be an OCR model, an image pipeline, an ASR model, an embedding model, or a video generator.

The ecosystem is broad, but the core strategy is simple:

1. Keep model, engine, and server concerns separate.
2. Start with the simplest runtime that meets your current need.
3. Choose a specialist model when the job is visual, audio, retrieval, or generative media rather than text.
4. Preserve portability through OpenAI-compatible interfaces where they fit, with a thin adapter for modality-specific differences.
5. Treat updates as controlled changes, not ad-hoc swaps.

Do that, and you get the best part of local AI: private, configurable intelligence you can run, test, and evolve on your own terms.

---

14 Appendix

14.1 Quick Command Cheatsheet

```
# === Ollama ===
# Verify current model tags in the Ollama library before pulling.
ollama pull qwen3-coder:8b
ollama run qwen3-coder:8b
ollama list
ollama rm qwen3-coder:8b
```

```
ollama serve

# === llama-server ===
llama-server -hf ggml-org/gemma-3-1b-it-GGUF --host 127.0.0.1 --port 8080
llama-server -m ./model.gguf --host 127.0.0.1 --port 8080

# === Podman + Ollama ===
podman run -d --name ollama -p 127.0.0.1:11434:11434 \
  -v ollama-data:/root/.ollama \
  ollama/ollama:<tested-tag>

# === Podman + Open WebUI ===
podman run -d --name open-webui -p 127.0.0.1:3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  -v open-webui:/app/backend/data \
  ghcr.io/open-webui/open-webui:<tested-tag>

# === Podman + LocalAI (NVIDIA) ===
# Verify the current CUDA-specific LocalAI image tag first.
podman run -d --name localai -p 127.0.0.1:8080:8080 \
  --device nvidia.com/gpu=all \
  -v localai-models:/build/models \
  localai/localai:<tested-nvidia-tag>

# === vLLM ===
uv pip install vllm
vllm serve meta-llama/Llama-3.3-8B-Instruct
```

14.2 Further Reading

- llama.cpp
- Ollama docs
- LocalAI docs
- vLLM docs
- SGLang docs
- Open WebUI
- Hugging Face model hub
- TypeSafe System One introduction
- TypeSafe: Introducing System One Models and Jev
- Laya open-weight model
- OpenJev open-weight model
- Decision-4B open-weight model
- LM Studio
- Jan
- Qwen3-VL
- Qwen3-ASR
- FLUX.2

- Wan2.2
- MLX
- Hugging Face Diffusers